What is Hadoop MapReduce?

Hadoop MapReduce is a **distributed programming model and processing framework** used to process **large datasets** across multiple machines in a Hadoop cluster.

It divides a large processing job into many smaller tasks, processes them in parallel on different machines, and then combines the results.

MapReduce is one of the core components of the Hadoop ecosystem.

Why Do We Need MapReduce?

Imagine you have a **5 TB log file**.

Processing it on a single computer would:

- Take a very long time.
- Consume too much memory.
- Be limited by a single CPU.

Instead, MapReduce:

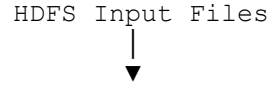
- Splits the file into blocks stored in HDFS.
- Processes each block on different machines in parallel.
- Combines the intermediate results into the final output.

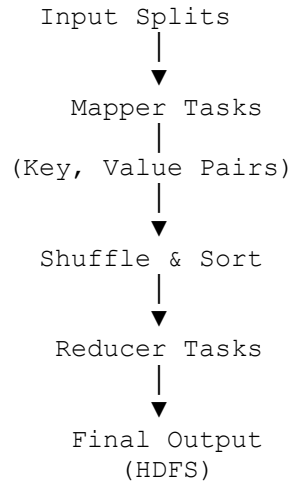
This enables Hadoop to process massive datasets efficiently.

MapReduce Workflow

A MapReduce job consists of four main phases:

1. Input Splitting
2. Map Phase
3. Shuffle and Sort Phase
4. Reduce Phase





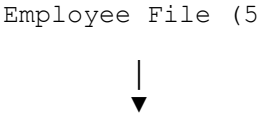
---

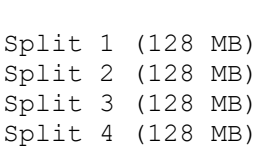
1. Input Splitting

Before processing begins:

- The input file is stored in **HDFS**.
- Hadoop divides the file into **input splits**.
- Usually, one split corresponds to one HDFS block.

For example:





Each split is processed independently by a mapper.

---

2. Map Phase

The **Mapper** processes one input split at a time.

It reads the data and produces **key-value pairs**.

Example

Input:

```
Apple Banana Apple Orange Apple
```

Mapper Output:

```
(Apple, 1)
(Banana, 1)
(Apple, 1)
(Orange, 1)
(Apple, 1)
```

The mapper performs filtering, parsing, and transformation, but **does not aggregate** the data.

---

3. Shuffle and Sort Phase

This is the most important phase in MapReduce.

Hadoop automatically:

- Groups values with the same key.
- Transfers data between nodes if necessary.
- Sorts the keys.
- Sends grouped data to reducers.

Example:

Mapper Output:

```
(Apple,1)
(Banana,1)
(Apple,1)
(Orange,1)
(Apple,1)
```

After Shuffle and Sort:

```
Apple  → [1,1,1]
Banana → [1]
Orange → [1]
```

This grouped data becomes the input to the reducers.

Note: The shuffle phase is often the most expensive part of a MapReduce job because it involves moving data across the network.

---

4. Reduce Phase

The **Reducer** processes each grouped key and its list of values.

Example:

Input:

```
Apple  → [1,1,1]
Banana → [1]
Orange → [1]
```

Reducer Output:

```
Apple   3
Banana  1
Orange  1
```

The reducer writes the final results back to HDFS.

Complete Word Count Example

Input:

```
Apple Banana Apple Orange Apple
```

Step 1: Map

```
Apple  → 1
Banana → 1
Apple  → 1
Orange → 1
Apple  → 1
```

Step 2: Shuffle & Sort

```
Apple  → [1,1,1]
Banana → [1]
Orange → [1]
```

Step 3: Reduce

```
Apple  → 3
```

```
Banana → 1
Orange → 1
```

Final Output:

```
Apple   3
Banana  1
Orange  1
```

Components of MapReduce

Component	Responsibility
Input Split	Divides the input data into smaller pieces.
Mapper	Processes input records and produces intermediate key-value pairs.
Shuffle & Sort	Groups values by key and sorts the keys before reduction.
Reducer	Aggregates values for each key and produces the final output.
HDFS	Stores the input and output data.
YARN	Allocates cluster resources and schedules MapReduce tasks.

Advantages of MapReduce

- Processes very large datasets.
- Executes tasks in parallel across multiple machines.
- Scales horizontally by adding more nodes.
- Provides fault tolerance by re-running failed tasks.
- Integrates tightly with HDFS and YARN.

Limitations of MapReduce

- Disk-based processing: intermediate results are written to disk.
- High latency for iterative workloads.
- Slower than Apache Spark for many analytics and machine learning tasks.
- More complex programming model than Spark's DataFrame API.

MapReduce vs Apache Spark

Feature	Hadoop MapReduce	Apache Spark
Processing	Disk-based	Primarily in-memory
Speed	Slower	Much faster for most workloads
Execution Model	Map → Shuffle → Reduce	DAG-based execution
Iterative Algorithms	Slow	Efficient
APIs	MapReduce API	RDD, DataFrame, Dataset, SQL
Best For	Large batch processing	Batch, streaming, machine learning, graph processing

---

Interview Questions

Q1. What is Hadoop MapReduce?

Hadoop MapReduce is a distributed programming model and processing framework that processes large datasets by dividing work into **Map** and **Reduce** tasks executed in parallel across a cluster.

Q2. What are the phases of MapReduce?

1. Input Splitting
2. Map
3. Shuffle and Sort
4. Reduce

Q3. What is the role of the Mapper?

The Mapper reads input records and converts them into intermediate **key-value pairs**.

Q4. Why is Shuffle and Sort important?

It groups all values with the same key, sorts the keys, and transfers grouped data to the appropriate reducers. Without this phase, reducers could not aggregate data correctly.

Q5. What is the role of the Reducer?

The Reducer processes each key and its associated values to produce the final aggregated result.

Q6. Why is Spark generally faster than MapReduce?

Spark keeps intermediate data **in memory** whenever possible and uses a **DAG-based execution engine**, reducing repeated disk I/O. In contrast, MapReduce writes intermediate results to disk between the Map and Reduce phases, which increases execution time.

Interview Summary

- **Hadoop MapReduce** is a distributed processing framework for large-scale batch data processing.
- A MapReduce job consists of **Input Splitting**, **Map**, **Shuffle and Sort**, and **Reduce** phases.
- The **Mapper** transforms input data into intermediate key-value pairs.
- The **Shuffle and Sort** phase groups values with the same key and sends them to the correct reducers.
- The **Reducer** aggregates the grouped values and writes the final output to HDFS.
- MapReduce is highly scalable and fault tolerant, but it is generally slower than Apache Spark because it relies heavily on disk I/O for intermediate results.